

Part A:

Procedure

1. Configure VirtualBox Network for Packet Capture
 - o Open VirtualBox → Kali VM → *Settings* → *Network*.
 - o Set Adapter 1:
Attached to: Bridged Adapter
Promiscuous Mode: Allow All
Cable Connected: 
 - o Save settings.
2. `sudo wireshark`
3. Select network interface `eth0`
4. Start Packet Capture
 - o Click on `eth0` to begin capturing packets.
 - o Generate some traffic (open websites, ping servers, or run curl commands).
5. Apply Display Filters for Analysis
 - o DNS Packets:
Dns
 - o HTTPS Packets:
Tls
 - o Packets from specific IP:
ip.addr == 18.161.216.37
 - o TCP Port 443:
tcp.port == 443

Part B: Capturing Login Credentials over HTTP

Step 1 – Setup Apache Web Server

```
sudo service apache2 start
sudo mkdir -p /var/www/html/testlogin
cd /var/www/html/testlogin
```

Step 2 – Create Login HTML Page

sudo nano login.html

```
html
<!DOCTYPE html>
<html>
<head>
  <title>Test Login Page</title>
</head>
<body>
  <h2>Login Form</h2>
  <form action="login.php" method="post">
```

```
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>
</body>
</html>
```

Save (Ctrl + O → Enter → Ctrl + X).

Step 3 – Create PHP Script

sudo nano login.php

```
<?php
$username = $_POST['username'];
$password = $_POST['password'];
echo "<h2>Login Attempt</h2>";
echo "Username: " . $username . "<br>";
echo "Password: " . $password . "<br>";
?>
```

Step 4 – Start/Enable Apache

sudo service apache2 status

sudo service apache2 start

Step 5 – Test Page

Browser:

http://127.0.0.1/testlogin/login.html

Step 6 – Capture with Wireshark

Open Wireshark → select lo (loopback) interface.

Apply filter:

http.request.method == "POST"Step 7 – Perform Login Attempt

Username: admin

Password: test123

Submit form.

LAB PROGRAM - 02

1) Prepare workspace & install required tools

```
mkdir -p ~/acq_demo && cd ~/acq_demo
sudo apt update
sudo apt install -y steghide zip unzip
```

2) Create benign victim files

```
mkdir -p ~/acq_demo/victim_docs
```

```
echo "this is a test secret." > ~/acq_demo/victim_docs/secret.txt

# harmless AV test string (optional, safe to keep)
echo
'X50!P%@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
' > ~/acq_demo/eicar.com.txt
```

3) Create the simulation script (core only)

Save this file as `~/acq_demo/simulate_malware.sh` — it creates a temp dir, copies the secret, creates a passworded zip, and stages it to `/tmp/exfil`:

```
cat > ~/acq_demo/simulate_malware.sh <<'EOF'
#!/bin/bash
set -euo pipefail
TMPDIR="$(mktemp -d /tmp/exfil.XXXXXX)"
trap 'rm -rf "${TMPDIR}"' EXIT
SRC="${HOME}/acq_demo/victim_docs/secret.txt"
cp "${SRC}" "${TMPDIR}/secret.txt"
zip -j -P pass123 "${TMPDIR}/payload.zip" "${TMPDIR}/secret.txt"
mkdir -p /tmp/exfil
cp "${TMPDIR}/payload.zip" /tmp/exfil/payload.zip
EOF
```

4) Make script executable and debug-run it

```
chmod +x ~/acq_demo/simulate_malware.sh
bash -x ~/acq_demo/simulate_malware.sh 2>&1 | tee
~/acq_demo/simulate_trace.log
```

5) Verify staged zip and extract to check password

```
unzip -l /tmp/exfil/payload.zip          # lists file inside
mkdir -p ~/acq_demo/recovered
unzip -P pass123 /tmp/exfil/payload.zip -d ~/acq_demo/recovered
cat ~/acq_demo/recovered/secret.txt
```

6) Ensure you have a cover JPEG (create one if needed)

If you already have an image, put it at `~/acq_demo/cover1.jpg`. If not:

```
sudo apt install -y imagemagick    # only if you need to create an
image
convert -size 800x600 xc:white ~/acq_demo/cover1.jpg
```

7) Embed the encrypted zip into the image with steghide

```
steghide embed -cf ~/acq_demo/cover1.jpg -ef /tmp/exfil/payload.zip -p
demo-pass
```

8) Conceal (rename/move) the stego-image (local-only)

```
mkdir -p ~/acq_demo/.hidden
mv ~/acq_demo/cover1.jpg ~/acq_demo/.hidden/.cover_hidden.jpg
```

9) Extract the payload from the stego-image and recover the secret

```
cd ~/acq_demo
steghide extract -sf .hidden/.cover_hidden.jpg -p demo-pass
# steghide will output payload.zip (or prompt); then:
unzip -P pass123 payload.zip -d ~/acq_demo/recovered_from_stego
cat ~/acq_demo/recovered_from_stego/secret.txt
```

10) Cleanup (optional)

```
rm -rf ~/acq_demo
rm -f /tmp/exfil/payload.zip
```

LAB PROGRAM - 03

Buffer Overflow Illustration using GCC/GDB/Linux Debugging Tools

```
sudo apt update
sudo apt install -y build-essential gdb
cd ~
nano overflow.c
```

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    Char buff[15];
    Int pass = 0;
    printf("\n Enter the password :\n");
```

```

    gets(buff);
    if(strcmp(buff, "thegeekstuff")){
        printf ("\n Wrong Password \n");
    }
    else{
        printf ("\n Correct Password \n");
        pass = 1;
    }
    if(pass)
        printf ("\n Root privileges given to the user \n");
    }
    return 0;
}

```

Ctrl + O → Enter → Ctrl + X

**gcc -g -O0 -fno-stack-protector -no-pie overflow.c -o overflow
./overflow**

1. Create Two Sample Files

nano file1.txt

nano file2.txt

2. Compare Files Using diff

diff file1.txt file2.txt

3. Use cmp for Byte-Level Comparison

cmp -l file1.txt file2.txt

4. Use vimdiff for Side-by-Side Colored View

vimdiff file1.txt file2.txt

5. Use meld (GUI Tool – Similar to Compare It)If you want a GUI comparison like

sudo apt update

sudo apt install meld -y

meld file1.txt file2.txt

6. Generate a Forensic Report

You can export results to a file:

diff -u file1.txt file2.txt > diff_report.txt